МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ

УНІВЕРСИТЕТ

РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з практичної роботи №2

з дисципліни «Високорівневі мови програмування та фреймворки»

на тему «Робота із форматом даних json, робота із запитами - метод fetch(). React.»

Виконав:

ст. гр. ПЗПІ-23-8

Козлов С. О.

Прийняв:

ст. в. каф. ПІ Саманцов О. О.

Харків 2026

2.1 Мета практичного завдання

Ознайомити студентів з базовими можливостями роботи із форматом даних json, робота із запитами - метод fetch(). React.

2.2 Індивідуальне завдання (Варіант 5)

- Рівень 1 (Завдання 5): Відображення зображення. Створіть файл HTML з елементом <script>. Використовуйте fetch для завантаження JSON-файлу з URL зображення. Створіть елемент <img> з URL зображення та додайте його на веб-сторінку.
- Рівень 2 (Завдання 5): Перевірка JSON. Напишіть програму, яка перевіряє, чи даний текст є дійсним JSON. Використовуйте вбудовані функції мови програмування або бібліотеки для перевірки JSON. Виведіть результат перевірки (дійсний JSON / недійсний JSON).
- Рівень 3 (Завдання 5): Розробіть веб-додаток інтерактивну дошку завдань, на якій користувачі можуть додавати завдання та присвоювати їм теги або мітки.
- Рівень 4 (Завдання 5): Доповніть ваш додаток можливістю реєстрації та управління профілем користувача (зміна пароля, інформації).

2.3 Хід вирішення

Повний вихідний код практичної роботи доступний у репозиторії на GitHub за посиланням: `github`

Для виконання практичної роботи було розроблено програмні рішення, що охоплюють базові принципи роботи з мережними запитами, обробку формату JSON та створення інтерактивного користувацького інтерфейсу за допомогою бібліотеки React.

Рівень 1. Відображення зображення

Опис реалізації: Для виконання цього завдання розроблено HTML-сторінку та підключено клієнтський JavaScript-сценарій. Основна мета – асинхронно завантажити дані з локального конфігураційного файлу image.json. Для цього використано сучасний метод fetch(). Процес обробки відбувається у вигляді ланцюжка промісів (promises). Після отримання успішної відповіді від сервера, текстові дані перетворюються на об'єкт JavaScript за допомогою вбудованого методу json(). Далі скрипт динамічно створює новий DOM-елемент <img> (за допомогою document.createElement), призначає йому отриманий URL у якості атрибута src та інтегрує його у визначений контейнер на сторінці.

Код файлу image.json:

```
{
  "imageUrl": "https://picsum.photos/300/200"
}
```

Код файлу script.js

```
fetch('image.json')
    .then(response => response.json())
    .then(data => {
        const imgElement = document.createElement('img');
        imgElement.src = data.imageUrl;
        imgElement.alt = "Зображення з JSON";
        document.getElementById('image-container').appendChild(imgElement);
    })
    .catch(() => console.log("ERROR"));:
```

Рівень 2. Перевірка JSON

Опис реалізації: Завдання полягає у створенні механізму валідації текстових даних на відповідність стандарту JSON. Реалізовано функцію validateJSON, яка приймає текстовий рядок. Всередині функції використовується блок перехоплення помилок try...catch. У блоці try виконується спроба декодувати рядок методом JSON.parse(). Якщо рядок має правильний синтаксис (відповідно до специфікацій об'єктів JavaScript), функція повертає повідомлення про дійсний JSON. Якщо ж у структурі є помилка (наприклад, відсутні подвійні лапки у ключів), JSON.parse() генерує виняток, який перехоплюється блоком catch, після чого функція сигналізує про недійсний формат.

Код реалізації (Node.js):

```
function validateJSON(jsonString) {
    try {
        JSON.parse(jsonString);
        return "дійсний JSON";
    } catch (error) {
        return "недійсний JSON";
    }
}

const validString = '{"name": "John", "age": 30}';
const invalidString = '{name: "John", age: 30}';

console.log(`Рядок 1: ${validateJSON(validString)}`);
console.log(`Рядок 2: ${validateJSON(invalidString)}`);
```

Рівень 3 та 4. Інтерактивна дошка завдань та управління профілем (React)

Опис реалізації: Додаток побудовано з використанням бібліотеки React на базі функціональних компонентів та хука стану useState. Логіку роботи розділено на дві основні частини, які відповідають вимогам 3-го та 4-го рівнів практичної роботи.

1. Система авторизації (Рівень 4): Реалізовано форму реєстрації та управління профілем користувача. Стан isAuthenticated контролює відображення інтерфейсу (форма входу або головна дошка). Функція handleRegister обробляє подію відправки форми, збираючи дані полів через об'єкт FormData. Після успішної "реєстрації" дані зберігаються у стані userProfile. Також додано функцію handleUpdateProfile, яка дозволяє користувачу змінити свій пароль, оновлюючи поточний стан профілю через spread-оператор.
2. Дошка завдань (Рівень 3): Реалізовано систему управління завданнями з можливістю додавання тегів. Дані завдань зберігаються у масиві об'єктів у стані tasks. Функція addTask відповідає за створення нового завдання: вона генерує унікальний ідентифікатор за допомогою Date.now(), формує об'єкт з текстом та тегом (з дефолтним значенням "Загальне", якщо тег не вказано), і оновлює загальний масив завдань. Відображення списку відбувається шляхом ітерації масиву методом map() у JSX-розмітці.

Стилізація додатку (файл App.css) виконана з використанням Flexbox-моделі для правильного позиціонування елементів та сучасних UI-рішень (градієнтний фон, тіні контейнерів box-shadow, плавні переходи стану transition).

Код файлу App.jsx:

```
import React, { useState } from "react";
import "./App.css";

export default function App() {
  const [isAuthenticated, setIsAuthenticated] = useState(false);
  const [userProfile, setUserProfile] = useState({
    username: "",
    password: "",
  });

  const [tasks, setTasks] = useState([]);
  const [newTaskText, setNewTaskText] = useState("");
  const [newTaskTag, setNewTaskTag] = useState("");

  const handleRegister = (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);
    setUserProfile({
      username: formData.get("username"),
      password: formData.get("password"),
    });
    setIsAuthenticated(true);
  };
```

```
  const handleUpdateProfile = (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);
    setUserProfile({
      ...userProfile,
      password: formData.get("newPassword"),
    });
    alert("Пароль успішно змінено!");
  };

  const addTask = () => {
    if (newTaskText.trim() === "") return;
    const newTask = {
      id: Date.now(),
      text: newTaskText,
      tag: newTaskTag || "Загальне",
    };
    setTasks([...tasks, newTask]);
    setNewTaskText("");
    setNewTaskTag("");
  };

  if (!isAuthenticated) {
    return (
      <div className="auth-container">
        <h2>Реєстрація (Рівень 4)</h2>
        <form onSubmit={handleRegister}>
          <input name="username" placeholder="Ім'я користувача" required />
          <input
            name="password"
            type="password"
            placeholder="Пароль"
            required
          />
          <button type="submit">Зареєструватися</button>
        </form>
      </div>
    );
  }

  return (
    <div className="board-container">
      <header>
        <h2>Вітаємо, {userProfile.username}!</h2>
          <form onSubmit={handleUpdateProfile} style={{ marginBottom: "20px"
}}>
          <input
            name="newPassword"
            type="password"
            placeholder="Новий пароль"
            required
          />
```

```
          <button type="submit">Оновити пароль</button>
        </form>
      </header>

      <main>
        <h3>Інтерактивна дошка завдань (Рівень 3)</h3>
        <div className="task-inputs">
          <input
            value={newTaskText}
            onChange={(e) => setNewTaskText(e.target.value)}
            placeholder="Опис завдання..."
          />
          <input
            value={newTaskTag}
            onChange={(e) => setNewTaskTag(e.target.value)}
            placeholder="Тег (напр. Терміново)"
          />
          <button onClick={addTask}>Додати завдання</button>
        </div>

        <ul className="task-list">
          {tasks.map((task) => (
            <li
              key={task.id}
              style={{
                border: "1px solid #ccc",
                margin: "10px 0",
                padding: "10px",
              }}
            >
              <strong>{task.text}</strong>
              <span className="task-tag">#{task.tag}</span>
            </li>
          ))}
        </ul>
      </main>
    </div>
  );
}
```

2.4 Результат виконання

Під час виконання практичної роботи було проведено тестування розроблених програмних рішень. Нижче наведено покрокову демонстрацію роботи додатку.

Рисунок 2.1 – Відображення випадкового зображення, отриманого за допомогою fetch-запиту з конфігураційного файлу (Рівень 1).

Рисунок 2.2 – Демонстрація динамічного оновлення даних: завантаження нового випадкового зображення після перезавантаження сторінки (Рівень 1).

Рисунок 2.3 – Результат виконання скрипта валідації JSON у терміналі (Рівень 2). Програма коректно розпізнає валідну структуру та виявляє синтаксичні помилки у недійсному форматі.

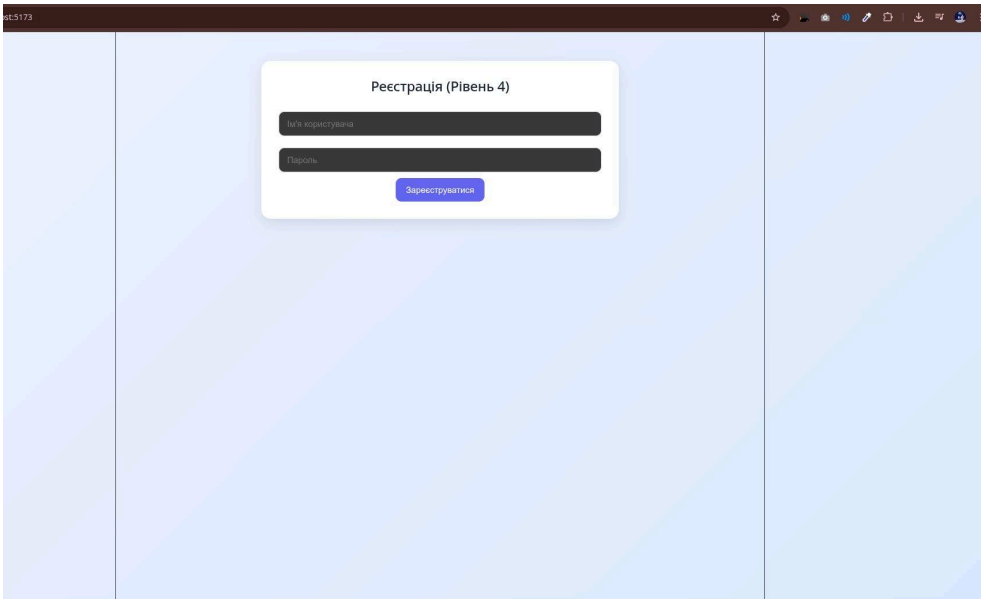

Рисунок 2.4 – Початковий стан компонента авторизації: порожня форма реєстрації користувача (Рівень 4).

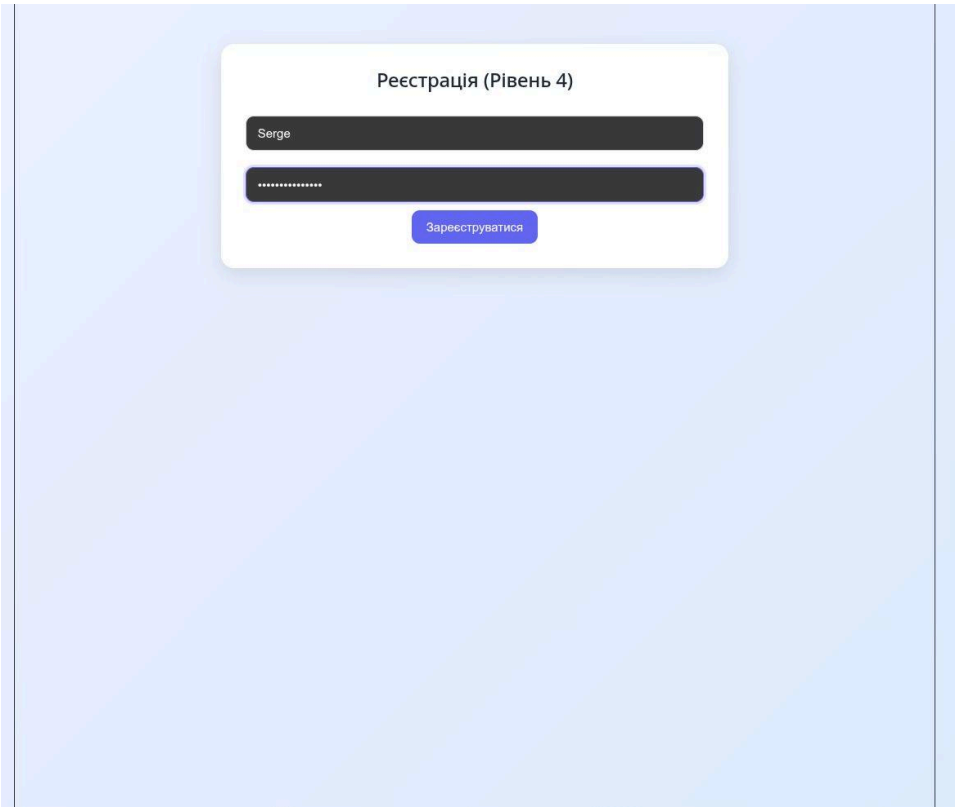

Рисунок 2.5 – Введення тестових даних (ім'я користувача та пароль) у форму реєстрації для отримання доступу до основного функціоналу додатку (Рівень 4).

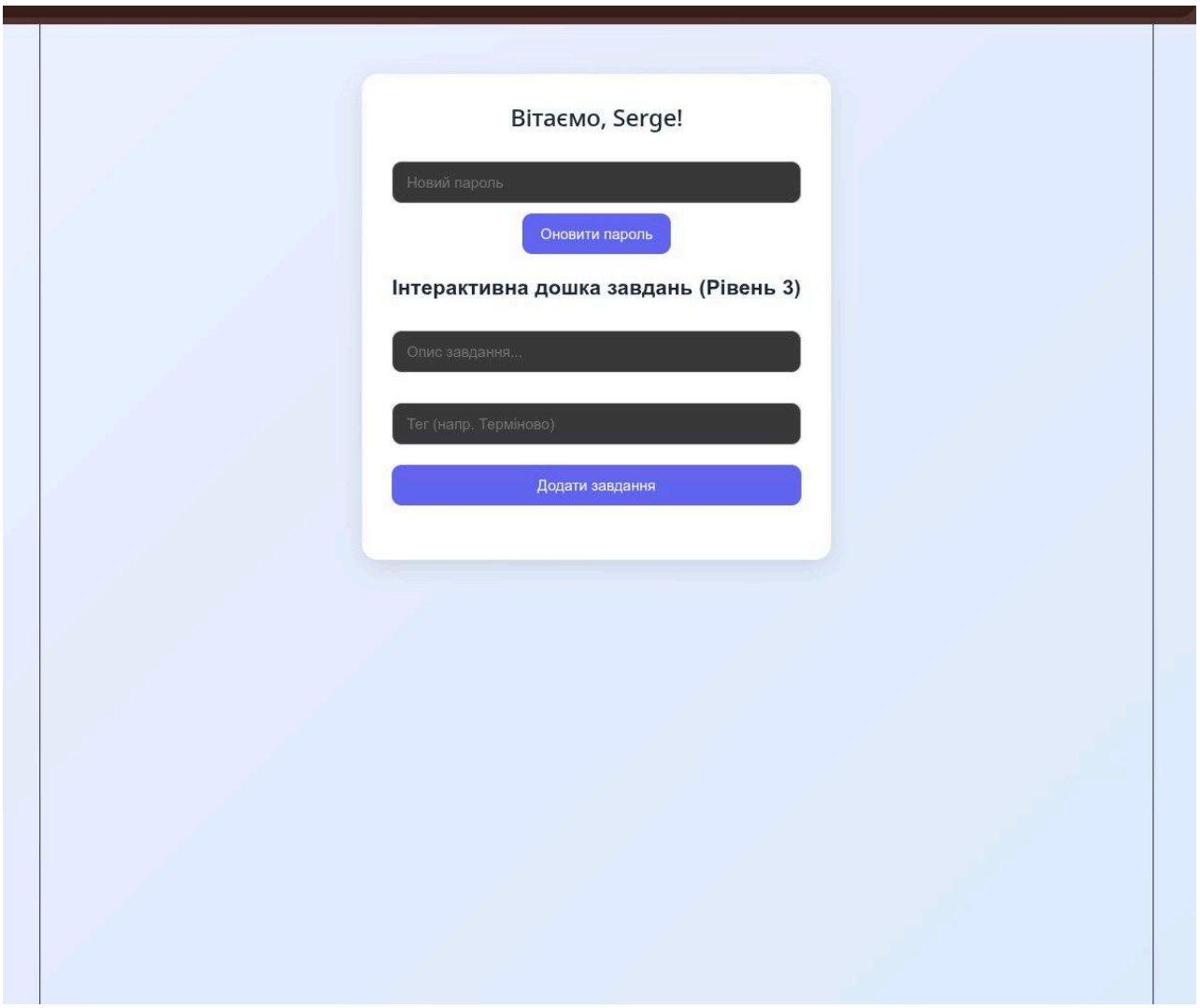

Рисунок 2.6 – Головний інтерфейс додатку після успішної авторизації. Зверху відображається персоналізоване привітання з іменем користувача, а нижче — порожня інтерактивна дошка завдань (Рівні 3 та 4).

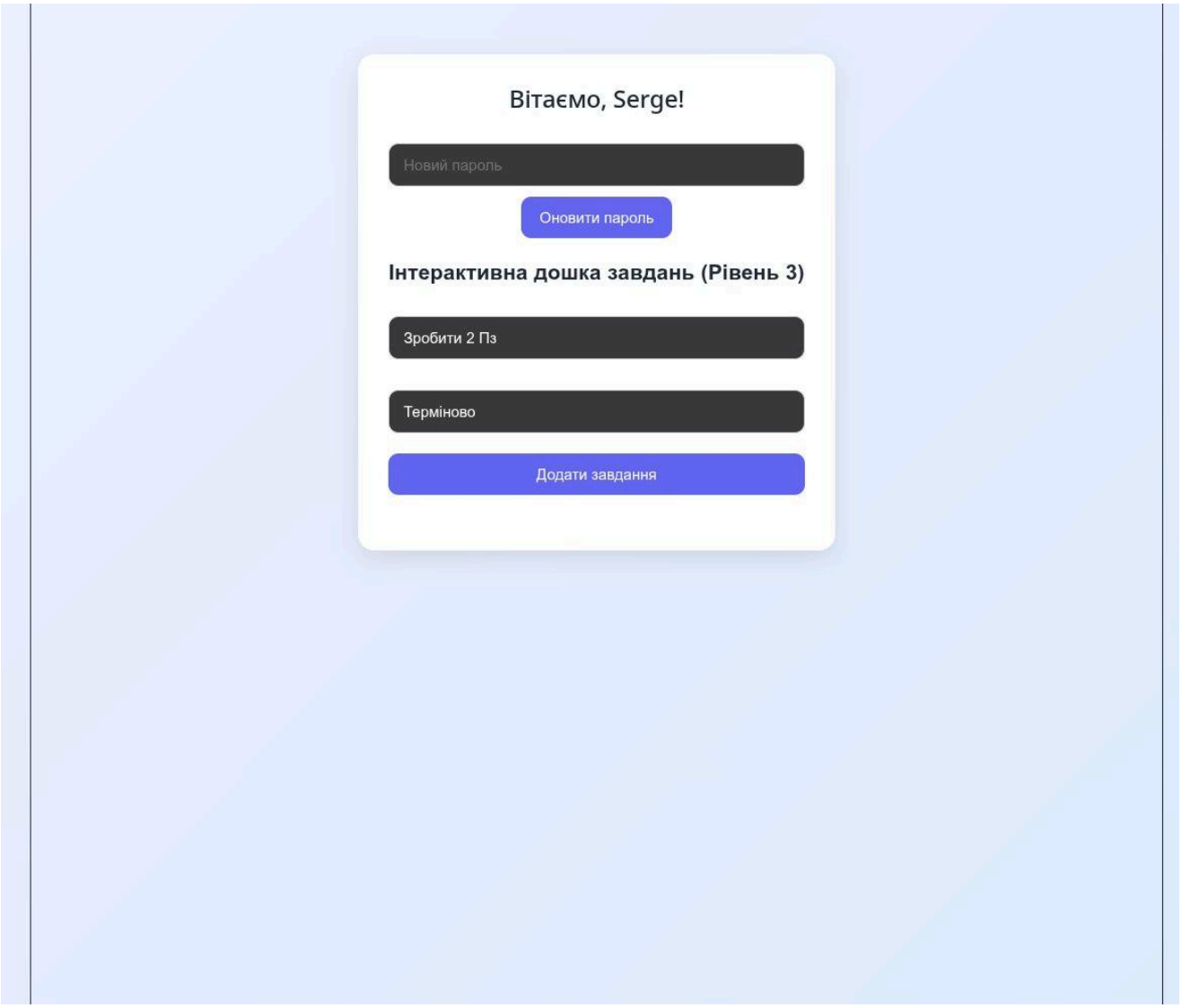

Рисунок 2.7 – Процес створення нового завдання: заповнення поля опису завдання (наприклад, "Зробити 2 ПЗ") та присвоєння йому користувацького тегу ("Терміново") (Рівень 3).

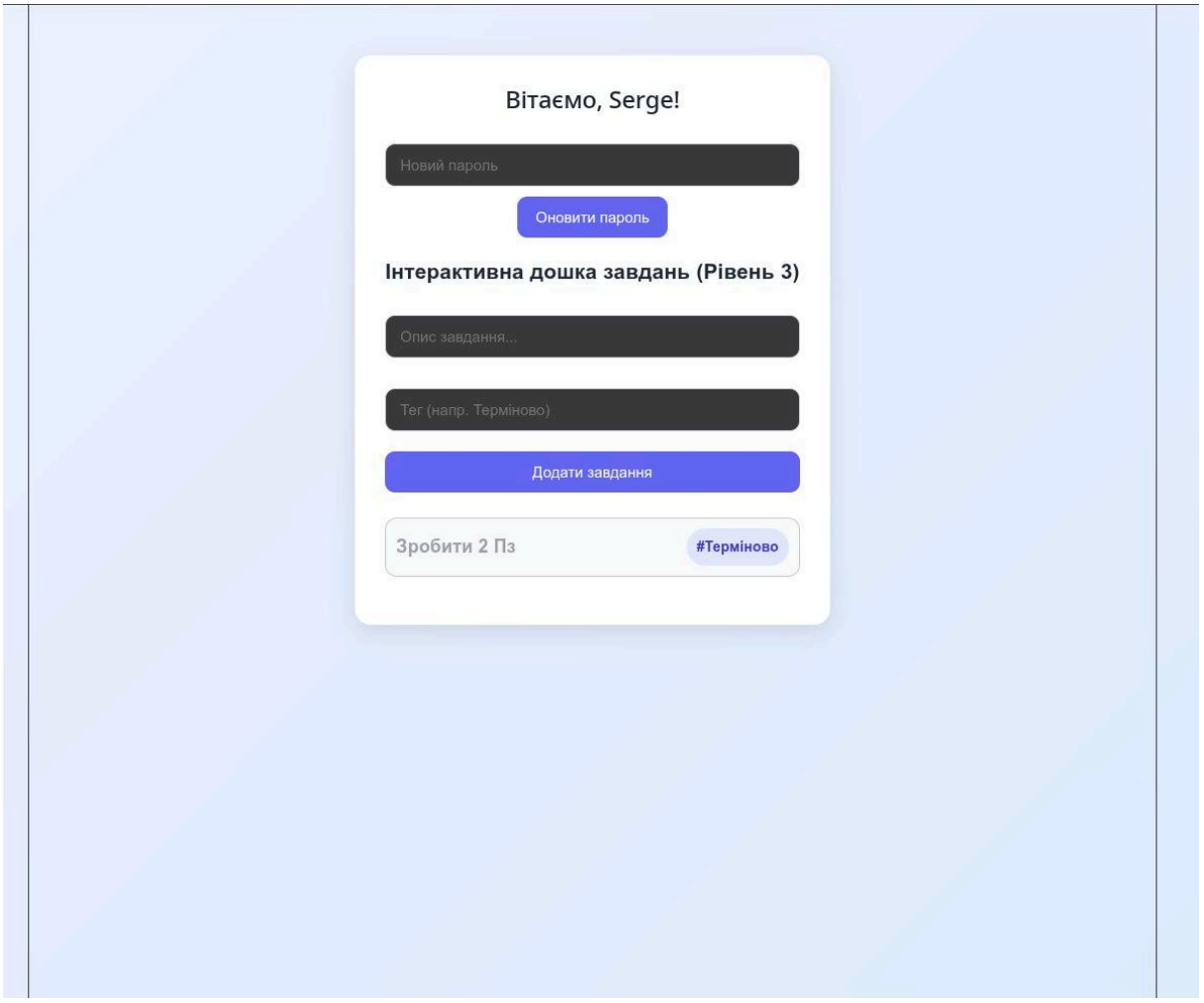

Рисунок 2.8 – Результат збереження стану: додане завдання успішно відображається у загальному списку разом із закріпленим тегом (Рівень 3).

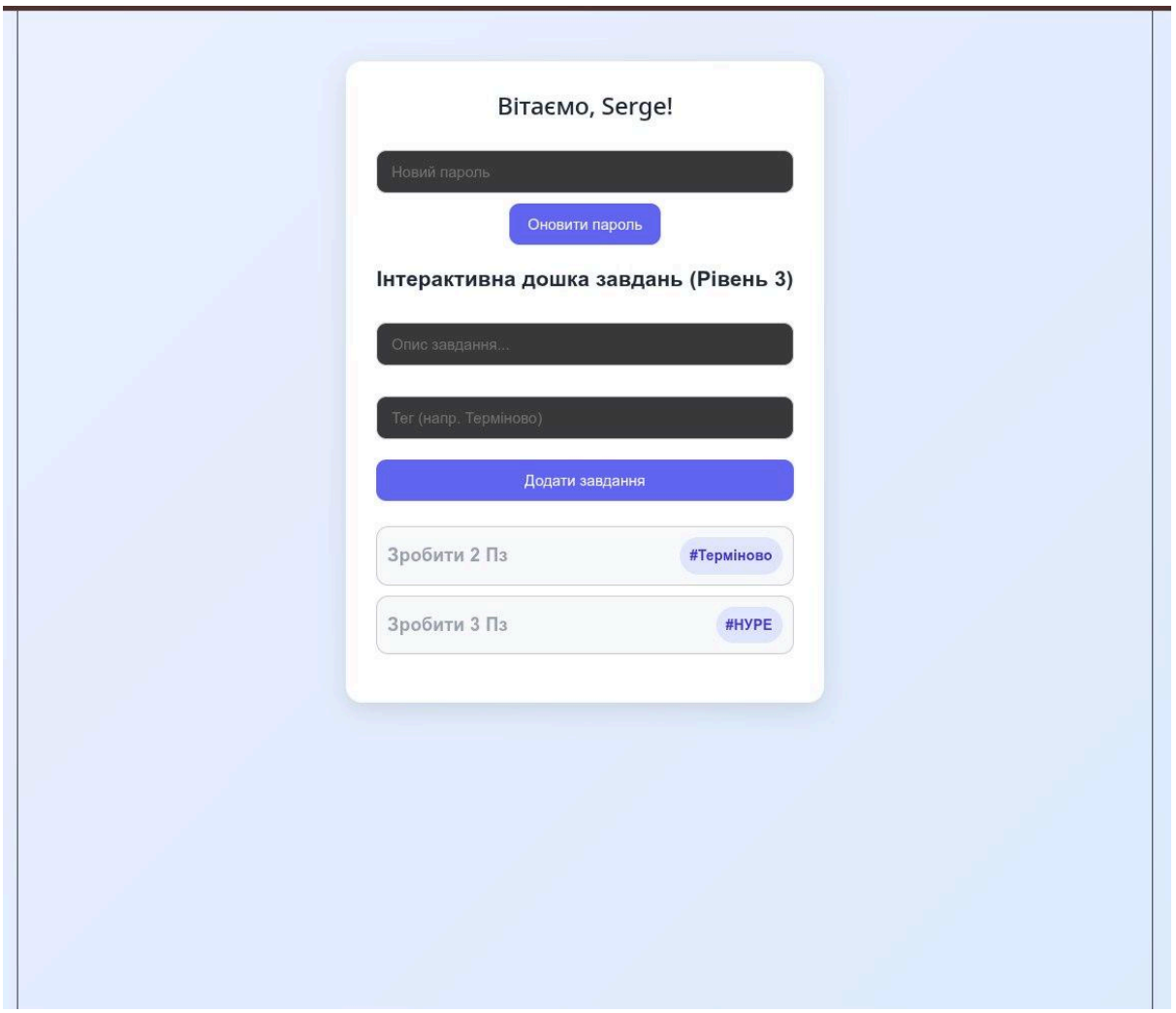

Рисунок 2.9 – Демонстрація роботи списку з декількома елементами: додано нові завдання з різними тегами, що підтверджує коректну роботу масиву об'єктів у стані компонента (Рівень 3).

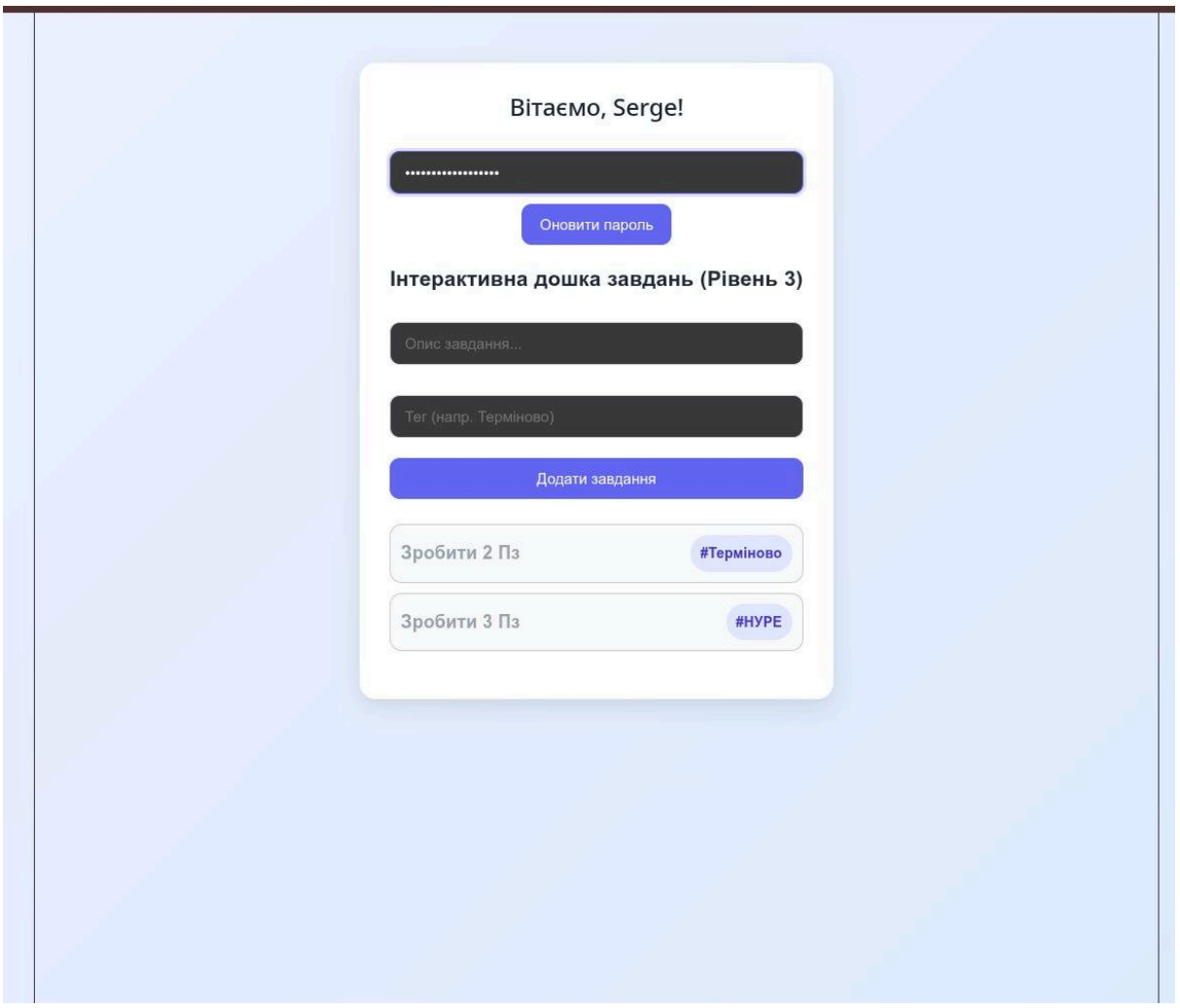

Рисунок 2.10 – Введення даних у форму оновлення профілю для зміни поточного пароля користувача (Рівень 4).

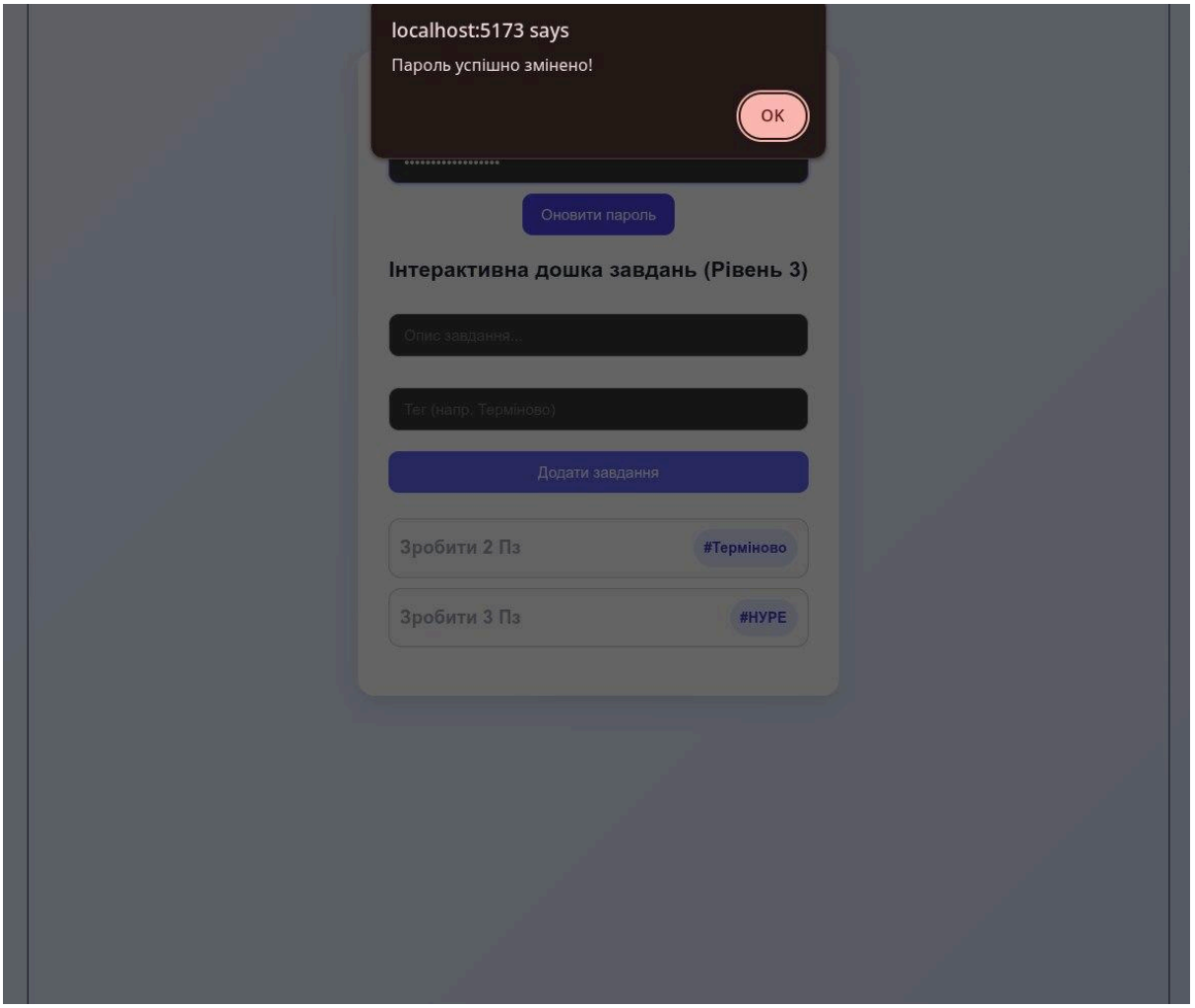

Рисунок 2.11 – Спливаюче вікно системи (alert), що підтверджує успішну зміну пароля у стані профілю (Рівень 4).

2.5. Висновки

Під час виконання практичної роботи було закріплено навички роботи з текстовим форматом JSON, який слідує за синтаксисом об'єкта JavaScript. Було успішно використано сучасний метод fetch() для відправлення мережних запитів та підвантаження інформації. Крім того, було розроблено веб-додаток на базі React, що реалізує інтерактивну дошку завдань із можливістю додавання тегів, а також систему реєстрації та управління профілем користувача.